WhatsApp Chatbot — Technical Handover

Tata Trusts Small Animal Hospital Mumbai (SAHM) — Laravel website & admin panel.

Document scope: the WhatsApp chatbot — architecture, conversation flows, data, admin management, configuration, testing and deployment. For developers taking over the project.

Contents

1. Overview
2. Architecture & key components
3. Message lifecycle (inbound & outbound)
4. Conversation flows
5. Direct appointment creation
6. Chatbot Management (admin-editable content)
7. Database schema
8. Routes & admin modules
9. Configuration / environment
10. Notifications
11. Local testing (simulator)
12. Deployment
13. Known limitations & design decisions

1. Overview

The chatbot is a **reactive WhatsApp assistant** for pet parents. It is served through **Fortius** (`waba.fortius.in.net`), a Business Solution Provider that proxies the **Meta WhatsApp Cloud API**. All send/receive payloads follow Meta's Cloud API format exactly.

- **Reactive only:** the user must message first (e.g. "Hi"); the bot then replies within WhatsApp's 24-hour customer-service window. Business-initiated messages would require Meta-approved templates (not currently used).
- **Stateful:** each conversation's current step + collected answers are stored per phone number.
- **Content-driven:** wording, options and menus are editable from the admin panel; the *flow structure* (step sequence + branching) lives in code.

2. Architecture & key components

Component	File	Responsibility
Receive webhook	`app/Http/Controllers/WhatsAppWebhookController.php`	GET verify handshake; POST parses inbound Meta payload → calls the bot engine.
Bot engine	`app/Support/WhatsAppBot.php`	The brain. Routes by conversation step, builds questions, handles all flows, creates appointments.
Send service	`app/Services/WhatsAppFortius.php`	Outbound messages: `sendText`, `sendButtons`, `sendList`, `sendImage`, `sendCtaUrl`. POSTs to `{base_url}/{version}/{phone_number_id}/messages`.
Conversation state	`app/Models/WhatsAppConversation.php`	Per-`wa_id` row: `step`, `data` (JSON: booking/report answers + history), `last_message_at`.
Editable content	`app/Models/ChatbotContent.php`	Admin-editable copy (questions, options, menu, FAQ, info). Static helpers the bot reads: `text()`, `options()`, `customSteps()`, `flowRows()`.
Delayed menu	`app/Jobs/SendWhatsAppMainMenu.php`	Re-shows the main menu a short delay after a flow completes (needs a queue worker for the real delay).
Appointment notifier	`app/Support/AppointmentNotifier.php`	Shared "new booking" SMS + emails, used by both website and WhatsApp bookings (null-safe).
Notification hooks	`app/Support/NotificationHooks.php`	Model `created` listeners → raise in-panel admin notifications.

Captured-data models

- `WhatsAppBookingRequest` — raw chat intake (audit record of each completed booking conversation).
- `AppointmentEnquiry` / `AppointmentUser` — the real appointment + client profile (shared with the website).
- `MedicalReportRequest` — "share my pet's reports" requests.
- `WhatsAppTeamQuery` — "Talk to our team" messages.

3. Message lifecycle (inbound & outbound)

INBOUND (USER → BOT)

```
user → WhatsApp → Fortius → POST  https://<domain>/webhooks/whatsapp
   → WhatsAppWebhookController@handle
   → parses entry[].changes[].value.messages[]  (text or interactive id)
   → WhatsAppBot@handle($waId, $profileName, $text, $interactiveId)
```

OUTBOUND (BOT → USER)

```
WhatsAppBot → WhatsAppFortius@send*  →  POST {base_url}/{version}/{phone_number_id}/messages
   (Authorization: Bearer <WHATSAPP_TOKEN>)  →  Fortius → WhatsApp → user
```

The webhook **GET** endpoint (`verify()`) is only used during webhook registration — it echoes `hub_challenge` when `hub_verify_token` matches `WHATSAPP_VERIFY_TOKEN`. Normal message traffic is all **POST**.

Step router

`WhatsAppBot@handle()` stores the conversation, then routes on `$convo->step` :

```
match ($convo->step) {
    'lead_reason'      => captureReason(...),      // Talk-to-team message
    'book_client_type' => chooseClientType(...),   // New / Existing choice
    'book_flow'        => continueBooking(...),     // Booking questions
    'report_flow'      => continueReport(...),      // Medical-report questions
    'book_day_custom'  => captureCustomDay(...),    // typed "pick a date"
    'book_heard_custom'=> captureCustomHeard(...),  // typed "how heard: Other"
    'book_review'      => reviewRouter(...),         // review + edit before confirm
    'book_edit'        => captureEdit(...),          // editing one field
    default            => routeMenu(...),            // idle: menu + taps
};
```

Reset words (`hi, hello, hey, menu, start, main menu, restart`) always return to the main menu. A thank-you / goodbye (`thanks, bye, that's all…`) ends the chat with a warm sign-off instead of re-showing the menu.

4. Conversation flows

4.1 Main menu

Greeting text + an interactive list. Items (editable via `menu.items`):

```
📅 Book appointment   🏥 Our services   📍 Timings & location
🚨 Emergency help     📄 Medical reports   💬 Talk to our team
```

Custom admin-added menu options (flow = `menu`) are appended; tapping one replies with its stored message.

4.2 Book appointment

First asks **New client** or **Existing client** (`book_client_type`). If a "New client" picks a number already registered, the bot asks them to choose "Existing client" instead.

NEW-CLIENT SEQUENCE

```
Pet parent name → Address → Email → Mobile → PIN
→ Pet name → Species → Sex → Breed → Colour → DOB/Age → Weight → Neutered?
→ Got it. What's the visit for?        (visit type)
→ Which speciality do you need?
→ What's the main concern for the visit?
→ [any custom questions]
→ Which day works best? → Preferred time?
→ Referred by? → How did you hear about us?
→ Review & Confirm
```

EXISTING-CLIENT SEQUENCE (BRANCHES ON UID)

```
Do you have your Patient ID / UID with us?   [Yes] [No]
├─ Yes → Patient ID/UID → Visit for → Speciality → [custom] → Day → Time → Review
└─ No  → Pet parent full name → Pet name → Mobile
         → Visit for → Speciality → [custom] → Day → Time → Review
```

The branch is driven by the `has_uid` answer. `bookingSteps($clientType, $answers)` is rebuilt on every step, so the step list adapts to the Yes/No answer while the already-answered prefix stays stable (the cursor remains valid).

SPECIALITY STEP

DB-driven: `bookableSpecialities()` returns a curated, ordered slug list (Cardiology, Dermatology, Dental Care, Diagnostic Imaging, Diagnostic Pathology, Oncology, Ophthalmology, Physiotherapy & Rehabilitation, Soft Tissue Surgery). The earlier per-doctor selection step was removed per client request (`doctorStep()` / `bookableDoctorsFor()` remain as dead code, re-enableable).

REVIEW, EDIT & CONFIRM

Before saving, a summary is shown (`sendReview`). The user can type a field name (e.g. "email", "day") to edit just that field (`reviewRouter` → `captureEdit`), or tap **Confirm & book** → `finaliseBooking` .

4.3 Medical reports

`menu_reports` → collects Client name, Pet name, Patient UID, Mobile, Report needed (+ any custom report-flow questions). Saved to `medical_report_requests` ; custom answers go to its `extra` JSON column. Confirms with a reference (`#MR-0001`) and "our team will contact you".

4.4 Our services

List of 6 rows + "View all on website": 3 pinned headline services (Emergency & IPD 24/7, OPD Consultation, Dialysis) + 3 specialities. Tapping a speciality (`svc_{id}`) sends a link to its page; the pinned items route to emergency info / specialities page / dialysis page. Capped at WhatsApp's 10-row list limit.

4.5 Timings, Emergency, FAQ, Talk to team

- **Timings** — address + hours (`info.hours` : IPD & Emergency 24×7; OPD 9:30 AM–7:30 PM) + emergency number + map.
- **Emergency** — phone-first help (`info.emergency`) + address + directions.
- **FAQ** — list of common questions (built-in + admin-added custom FAQs); each answers with editable text.
- **Talk to team** — captures a free-text message → `whatsapp_team_queries` + emails Customer Care.

5. Direct appointment creation

On **Confirm & book**, `finaliseBooking` → `createAppointmentFromBooking` :

1. Saves the raw intake (`WhatsAppBookingRequest`) for audit.
2. Find-or-creates the `AppointmentUser` keyed by mobile (mobile falls back to the WhatsApp number when not collected — e.g. the UID branch).
3. Creates an `AppointmentEnquiry` with `source = 'whatsapp'` , a per-source running reference (`WA-0001`), default status, and the chosen speciality. Chat-only extras (breed, how-heard, custom answers, patient UID…) are folded into the `reason` field.
4. Runs `AppointmentNotifier::notifyNewBooking()` — the same tentative-booking SMS + owner email + admin email as a website booking (null-safe; sends even without a firm date).
5. Replies with a confirmation + `WA-` reference. **No website login link.**

WhatsApp appointments appear in the admin **WhatsApp Appointments** tab and the reports, and raise a "New WhatsApp Appointment" in-panel notification.

6. Chatbot Management (admin-editable content)

Admin section **Chatbot Management** (`manage-chatbot.*` , permission `chatbot-management.view/edit`) edits rows in `chatbot_contents` . The bot overlays them onto its code steps via `WhatsAppBot@overlaySteps()` ; if a key is missing/inactive the built-in default is used, so the flow never breaks.

WHAT ADMINS CAN MANAGE

- **Question / message wording** for every step (keys like `book.reason` , `book.has_uid` , `report.mobile`).
- **Answer options** for choice steps + their order (drag to reorder), enable/disable.
- **Menu** greeting + items, **Info** blocks (hours, emergency), **FAQ** answers.
- **Add new items**: custom Booking / Medical-Report questions (text or multiple-choice), FAQ entries, Main-menu options — with a **Priority** field for ordering and a WhatsApp-style editor + live preview.

> **Protected by design:** the step *sequence* and *branching anchors* stay in code. Branch-driving options (New/Existing, Yes/No UID, Pick-a-date, How-heard "Other") keep their option IDs in code so renaming them in the panel can't break the flow. Question *order* is not panel-editable (the two booking variants make a single order field unreliable) — devs change it in code.

7. Database schema

Table	Purpose	Key columns
`whatsapp_conversations`	Per-user chat state	wa_id, step, data (JSON), last_message_at
`whatsapp_booking_requests`	Raw booking intake (audit)	wa_id, client_type, parent_name, pet_name, patient_uid, …, converted_at
`appointment_enquiries`	Real appointments (web + WhatsApp)	source, ref_no, speciality, preferred_doctor, appointment_status_id, …
`appointment_users`	Client profiles	mobile, name, email, address, pincode
`medical_report_requests`	Report requests	client_name, patient_name, patient_uid, mobile, report_name, extra (JSON)
`whatsapp_team_queries`	Talk-to-team messages	wa_id, name, message, status
`chatbot_contents`	Admin-editable copy	section, flow, content_key, field_key, body, options (JSON), is_choice, is_active, is_protected, sort_order

8. Routes & admin modules

Route	Purpose	Permission
`GET/POST /webhooks/whatsapp`	Verify handshake / inbound messages	public
`manage-whatsapp-appointments.*`	WhatsApp appointments list/detail/status	whatsapp-appointments.*
`manage-whatsapp-queries.*`	Talk-to-team inbox	whatsapp-queries.view
`manage-medical-reports.*`	Medical-report requests inbox	medical-reports.view
`manage-chatbot.*`	Chatbot Management (content)	chatbot-management.view/edit

Permissions are seeded by `ModulePermissionsSeeder` and assigned to Super Admin + Admin (Front Office additionally for the inboxes).

9. Configuration / environment

```
# WhatsApp — Fortius (Meta Cloud API proxy)
WHATSAPP_BASE_URL=https://waba.fortius.in.net
WHATSAPP_API_VERSION=v18.0
WHATSAPP_PHONE_NUMBER_ID=<id of the connected number from Fortius>
WHATSAPP_TOKEN=<API bearer token>
WHATSAPP_VERIFY_TOKEN=<webhook-registration handshake secret>
WHATSAPP_CHAT_NUMBER=<number the website "Chat with us" button opens, digits+country code>
```

Config is bound in `config/services.php` under `services.whatsapp`. SMS (MessageIndia) + mail are configured separately and used by `AppointmentNotifier`.

> **Number status:** verify the connected number via the Fortius API console → *Get Channels* (version `v18.0`). A `+1 555…` `display_phone_number` is Meta's **test** number — it only replies to a small allowlist and **cannot be used for production**. A real business number must be onboarded (its own `phone_number_id`) before go-live.

10. Notifications

`NotificationHooks` registers model `created` listeners that call `AdminNotification::raise()`:

- `AppointmentEnquiry` → "New WhatsApp Appointment" (source-aware link) or "New Appointment Booking" (website).
- `MedicalReportRequest` → "New Medical Report Request".
- `WhatsAppTeamQuery` → "New WhatsApp Query".

Shown via the in-panel bell (AJAX polling). Emails for bookings/queries go out via `AppointmentNotifier` / the bot's email helpers; every send is recorded in `communication_logs` (visible under Reports → Communication).

11. Local testing (simulator)

No WhatsApp credentials needed — `php artisan whatsapp:simulate` fakes an inbound message and prints the bot's replies:

```
# Start fresh and open the menu
php artisan whatsapp:simulate --fresh --from=919812345678 "hi"

# Continue the same conversation (state persists per --from)
php artisan whatsapp:simulate --from=919812345678 "menu_book"
php artisan whatsapp:simulate --from=919812345678 "client_existing"
# … tap ids (menu_book, client_new) or type free text/button titles
```

12. Deployment

```
git pull
composer install --optimize-autoloader --no-dev
php artisan migrate --force
php artisan db:seed --class=ModulePermissionsSeeder --force
php artisan db:seed --class=ChatbotContentSeeder --force              # fresh content (firstOrCreate)
php artisan db:seed --class=ChatbotContentUpdateOct2026Seeder --force # content tweaks on already-seeded envs
php artisan config:cache && php artisan route:cache && php artisan view:cache
```

Content note: `ChatbotContentSeeder` uses `firstOrCreate` (never clobbers admin edits), so content changes for an *already-seeded* environment are applied via `ChatbotContentUpdateOct2026Seeder`. The bot also falls back to code defaults for any missing content row, so it works even unseeded (the admin panel would just be empty).

Queue: the delayed main-menu job + queued mail need a worker (`php artisan queue:work`) or set `QUEUE_CONNECTION=sync` .

13. Known limitations & design decisions

- **Flow structure in code:** step order & branching live in `WhatsAppBot::bookingSteps()` / `reportSteps()` . Content (wording/options) is in the DB. This keeps branching safe while letting non-developers manage copy.
- **WhatsApp list limit:** interactive lists allow max 10 rows — the services and speciality lists are trimmed accordingly (rest via "View all on website").
- **Test vs live number:** the bot replies to the whole public only on a real connected business number; the sandbox number is allowlist-only.
- **Reactive only:** the bot cannot initiate a conversation; proactive/>24h messages would need approved templates.
- **Dead code kept intentionally:** `doctorStep()` / `bookableDoctorsFor()` remain for easy re-enable of per-doctor selection.
- **Debug log:** the webhook has a temporary `Log::info('WhatsApp inbound payload', …)` — remove before production (it logs every user's number + message).

End of document. Generated as a developer handover for the SAHM WhatsApp chatbot.